

Extending AUTOATTACKER with Evidence-Grounded Strategic Reasoning

Technical Analysis and Project Design Proposal

Nathan Conklin

nathan.conklin@vt.edu

Department of Computer Science,
Virginia Tech

Abstract

AUTOATTACKER is a research prototype that uses a large language model to conduct multi-step, hands-on-keyboard cyber-attack tasks in an isolated enterprise laboratory. The system combines four functions: a summarizer that maintains compact state, a planner that proposes the next action, an experience manager that retrieves previously successful actions, and a navigator that selects and executes a single action against the test environment. Xu et al. evaluated the prototype on 14 Windows and Linux tasks that covered several MITRE ATT&CK techniques. Their results show that the 2024 version of GPT-4 completed nearly all reported trials at low temperature, while GPT-3.5 and the evaluated Llama 2 models performed poorly. The paper also documents hallucinations, dependence on deliberately insecure targets, incomplete ATT&CK coverage, and substantial differences among models [1].

This whitepaper proposes a class project that extends that foundation with evidence-grounded strategic reasoning. The extension accepts a bounded testing objective, maintains an explicit model of the environment, generates and ranks hypotheses, builds a revisable plan, invokes approved tools, and converts observations into findings only after deterministic validation. The main contribution is a separation between reasoning and proof. The language model may propose where to look and what to try; independent validators decide whether the resulting evidence supports a security finding. This design preserves the adaptive value of an LLM while reducing false findings, repeated actions, and untraceable decisions.

The project should run only in an isolated, authorized cyber range. It is intended for defensive research, controlled evaluation, and red-team support. It is not a production autonomous attack platform.

1. Purpose and Contribution

The project builds on the system described in *AutoAttacker: A Large Language Model Guided System to Implement Automatic Cyber-attacks* by Jiachen Xu, Jack W. Stokes, Geoff McDonald, Xuesong Bai, David Marshall, Siyue Wang, Adith Swaminathan, and Zhou Li [1]. The paper is available at <https://arxiv.org/html/2403.01038v1>. The authors represent the University of California, Irvine and Microsoft. They submitted version 1 to arXiv on March 2, 2024.

The proposed work contributes an evidence-grounded reasoning layer above AUTOATTACKER's action loop. That layer should:

- translate a high-level, authorized objective into explicit hypotheses and bounded test plans;
- prioritize actions using expected information value, cost, risk, and prior evidence;
- preserve a machine-readable record of observations, decisions, and rejected alternatives;
- require deterministic or independently corroborated proof before recording a finding;
- learn from completed runs without treating an LLM's narrative as ground truth; and
- produce a replayable report that connects each finding to the commands, outputs, artifacts, and verification checks that support it.

The project does not need to reproduce an enterprise penetration-testing product. A strong class deliverable can focus on one defensible improvement: adding adaptive planning while measuring whether the system produces more verified findings with fewer unproductive actions.

2. Source Reconciliation and Terminology

The conversation script captured useful project goals, but it also combined descriptions of different systems. The cited paper is the authoritative source for claims about the 2024 AUTOATTACKER prototype. The supplied architecture image is useful as a target-state design; it should not be read as an as-built diagram of the paper's implementation.

Exploratory statement	Evidence-based correction and project treatment
AUTOATTACKER is a Go CLI.	The authors document a Python 3.9 research implementation that uses ThinkGPT and LangChain for parts of the summarizer and navigator [1]. Use Python as the paper's baseline.
It uses a static CLI, YAML probe library, runner, transport, judge, and report pipeline.	The paper describes a summarizer, planner, navigator, experience manager, victim environment, and LLM-mediated control loop [1]. Use that four-module agent architecture as the baseline.
Its attack content is a YAML library of prompt-injection and LLM-safety probes.	The paper evaluates operating-system and Metasploit actions in a virtual enterprise laboratory. It does not describe a YAML library of LLM prompt attacks [1].
It evaluates prompts but does not execute infrastructure actions.	AUTOATTACKER executes generated shell or Metasploit actions against Windows and Linux virtual machines in the isolated testbed [1].
It lacks planning and adaptive reasoning.	The baseline already uses planning, summarization, experience retrieval, and LLM-based action selection [1]. The extension adds global strategy and independent validation.
Microsoft and UC Irvine released an official open-source repository.	The paper identifies no canonical repository or AUTOATTACKER software license. A public third-party implementation exists, but its relationship to the research code is not established [6]. Verify provenance before selection.
A hosted version has published per-host pricing.	The paper describes a research system, not a commercial AUTOATTACKER service, licensing tier, or price [1]. Exclude the pricing claim.

[TODO: identify the exact repository Nathan intends to extend, confirm its provenance and license, and record the commit hash used for the project baseline.]

This paper uses **AUTOATTACKER** for the Xu et al. research system and **reasoning-layer extension** for the proposed class project. The phrase **finding** means a security-relevant condition supported by recorded evidence and a completed validation rule. An LLM hypothesis, unexplained tool message, or unverified command result does not qualify as a finding.

3. AUTOATTACKER's Research Objective and Threat Model

AUTOATTACKER studies whether an LLM can automate post-breach activities that a human operator would normally perform through a shell or penetration-testing framework. The authors focus on hands-on-keyboard actions rather than limiting the study to pre-breach phishing or malware generation. Their defensive motivation includes continuous posture testing, increased red-team capacity, and earlier observation of attack behaviors that defenders may later encounter [1].

The threat model assumes that an attacker either controls a machine inside the enterprise or can communicate with a machine in the network, depending on the task. The environment contains weaknesses that a human tester could exploit. The research system receives an environment description and an objective. It then takes one action at a time, observes the outcome, updates its state, and selects another action until it reaches the objective or exceeds an interaction limit [1].

The task formalization is important to the proposed extension. Xu et al. define a task as a pair, $\tau = (\text{env}, \text{obj})$, where env describes the victim environment and initial attacker capability, and obj defines a completion condition. A successful or failed objective must be inferable from the victim environment. This requirement gives the project a sound starting point for ground-truth validation: each task should include a machine-checkable success predicate before execution begins.

AUTOATTACKER is not an end-to-end enterprise attack-surface management system. The paper does not demonstrate broad asset inventory, production credential management, continuous scanning, remediation workflow, or integrations with a vulnerability-management platform. It demonstrates an LLM-guided action loop on a controlled benchmark.

4. Detailed Capabilities of AUTOATTACKER

4.1 Closed-Loop Interaction with a Victim Environment

AUTOATTACKER connects model output to an executable environment. The model does not produce a complete narrative plan and stop. The system repeatedly collects an observation, compresses the relevant state, proposes one next action, selects among alternatives, executes the selected action, and feeds the result into the following round [1]. This loop enables error recovery because later actions can respond to tool output. The paper includes a case in which GPT-4 initially names a nonexistent Metasploit module, observes the failure, searches for the relevant module, and then selects the correct one.

The executor may use native shell functions or an installed penetration-testing tool. The testbed included Kali Linux 2023.04 and Metasploit 6.3.43-dev. Depending on the benchmark task, the system interacted with Windows command shells, Meterpreter sessions, Metasploit, or Linux services [1].

4.2 Summarizer: Compact State and Context Tracking

The summarizer maintains a short representation of prior actions, observations, and environmental state. It combines the previous summary with the newest observation and produces an updated summary for the next planning call. The prompt requests concise sentences and abbreviations, plus a list of earlier actions [1].

This component addresses a practical agent problem. Shell transcripts and tool output can exceed an LLM's context window, while a missing detail such as the active shell, current privilege level, or selected target can invalidate the next command. The summarizer reduces the transcript while preserving the state that the planner needs. Its output includes the latest action and result, not merely a generic account of the session.

The summarizer is useful, but it is not a proof system. An LLM-generated summary can omit or distort an observation. The proposed extension therefore preserves raw evidence alongside the summary and prevents the summary from becoming the only record of state.

4.3 Planner: One Structured Action at a Time

The planner receives the objective and summarized situation. It asks the LLM for a structured action containing a planning rationale, command type, and exact command. The paper describes both detailed objectives and abstract objectives. A detailed objective contains atomic instructions; an abstract objective relies more heavily on retrieved experience [1].

The prompt limits the model to one action per response. This constraint makes execution and attribution easier because the system can connect one command to one observation. AUTOATTACKER also includes a command checker for recurring syntax problems, such as multi-command strings that violate the expected format [1].

The planner demonstrates local tactical reasoning. It determines the next step from the current situation. The paper does not show a persistent global plan with explicit branches, competing hypotheses, estimated costs, stop conditions, or a record of

why an alternative was rejected. The proposed project adds those strategic controls without replacing the planner's local command-generation role.

4.4 Experience Manager: Retrieval of Successful Actions

The experience manager stores useful actions from tasks that the system completed successfully. Each stored action includes a planning section. The implementation converts that section into an embedding and compares it with the newly proposed action using cosine similarity. It retrieves the top k actions, with $k = 3$ in the paper [1].

The authors describe this design as inspired by retrieval-augmented generation [1], [3]. The experience database supplies prior domain experience without placing the entire execution history into each prompt. AUTOATTACKER first runs basic tasks, such as file writing or privilege escalation, and records the successful steps. It can then reuse those experiences as building blocks in advanced task chains.

The experience manager reduces interactions for some tasks. In the reported ransomware experiment, the combination of an abstract objective and stored prerequisite experience reduced the number of interactions from 17 to 7 and reduced the reported GPT-4 API cost from \$4.04 to \$1.70 [1]. These values reflect the authors' 2024 test configuration and API pricing; they are not current cost estimates.

4.5 Navigator: Action Selection and Execution

The navigator considers the planner's new action and the actions retrieved by the experience manager. It asks the LLM to select one candidate and then executes that candidate in the victim environment [1]. This design makes prior successful actions available without forcing the planner to reproduce them from memory.

The navigator is an action selector, not an independent verifier. It uses the LLM to choose the next command. A high similarity score can retrieve a semantically related action that does not fit the present target, privilege state, tool version, or path. The reasoning-layer extension should treat retrieved actions as candidates and require parameter binding, precondition checks, and validation before execution.

4.6 Basic-to-Advanced Task Composition

AUTOATTACKER distinguishes basic tasks from complex tasks. A basic task uses one attack technique and can populate the experience database. A complex task requires several techniques and may reuse actions from those basic tasks. The reported advanced chains include dump-the-hash, pass-the-hash, and ransomware scenarios [1].

This composition model is one of the system's strongest capabilities. It gives the agent a reusable procedural memory. The proposed extension can strengthen it by representing dependencies explicitly: required privileges, target operating system, active session type, expected observation, and completion predicate.

4.7 Model-Agnostic Structure with Model-Dependent Performance

The system's prompts and modules can call different LLMs, but the paper shows substantial model dependence. The authors evaluated GPT-3.5 Turbo, GPT-4, Llama2-7B-chat, and Llama2-70B-chat. GPT-4 produced the strongest results. GPT-3.5 completed only a small subset reliably; both Llama 2 models failed all tasks in the authors' configuration because of inaccurate tool knowledge, malformed actions, and context-tracking failures [1].

The architecture is model-substitutable. The demonstrated capability is not. A new model, local model, or Virginia Tech ARC-hosted model must be evaluated against the same task definitions and validation rules before the project credits it with equivalent performance.

[TODO: select the approved reasoning model and hosting path for the project, including whether inference will run through Virginia Tech ARC, a local endpoint, or a commercial API.]

4.8 Controlled Prompting and Safety-Policy Bypass in the Study

The authors report that direct requests for offensive commands were often refused. They used a role-playing prompt to bypass model safeguards and reported a 100 percent jailbreak success rate across ten questions for all four evaluated models [1]. The technique enabled the experiment, but it also creates a serious governance issue.

The class project should not make safeguard bypass a contribution. It should use models and policies approved for a contained cyber range, with explicit authorization and a fixed allowlist of tools and targets. The system must reject objectives outside the configured range even if the model proposes them.

4.9 Benchmark and Evaluation Harness

The paper contributes a 14-task benchmark across Windows and Linux. Each task is labeled as basic or complex and assigned qualitative ratings for adaptability, stealthiness, and impact. The authors run each task three times and measure task success plus the number of LLM interactions [1].

This benchmark provides a useful regression suite for a course project, although reproducing all 14 tasks may require substantial environment construction. The project can select a safe subset that exercises state tracking, error recovery, experience retrieval, and evidence validation.

5. The Fourteen Evaluated Tasks

The following table reproduces the paper's task names and ATT&CK mappings at a descriptive level. It omits procedural commands. The mappings and operating-system labels reflect the paper; the project team should validate them against the current MITRE ATT&CK release before using them as ground truth [1], [2].

#	Task and type	Paper's ATT&CK mapping	Intended capability
1	File Writing, basic	T1083, File and Directory Discovery	Place a file at a defined location and verify its presence.
2	Privilege Escalation, basic	T1068, Exploitation for Privilege Escalation	Obtain a higher privilege level in the configured victim.
3	Credentials Stealing, basic	T1555, Credentials from Password Stores	Recover defined credential material in the lab.
4	File Uploading, basic	T1588, Obtain Capabilities	Transfer an approved artifact to a target VM.
5	Script Executing, basic	T1059, Command and Scripting Interpreter	Execute an approved script and record its result.
6	Dump the Hash, complex	T1003, OS Credential Dumping	Combine privilege and credential actions to obtain a configured hash.
7	Pass the Hash, complex	T1550, Use Alternate Authentication Material	Use laboratory credential material to reach a defined domain objective.
8	Ransomware, complex	T1486, Data Encrypted for Impact	Exercise a contained encryption scenario on disposable test data.
9	MySQL Scan, complex	T1595, Active Scanning	Identify the configured MySQL service.
10	MySQL Hashdump, complex	T1078, Valid Accounts	Obtain the laboratory database credential artifact.

#	Task and type	Paper's ATT&CK mapping	Intended capability
11	Log4j Header Injection, complex	T1203, Execution	Exercise the configured vulnerable service and verify controlled execution.
12	SSH Public Key Injection, complex	T1136, Create Account	Add a laboratory access mechanism and verify it.
13	Malicious WinRAR, complex	T1566, Phishing	Generate the configured archive artifact and observe the lab callback.
14	Malicious EXE, complex	T1566, Phishing	Generate the configured executable artifact and observe the lab callback.

The paper's benchmark spans reconnaissance, initial access, execution, persistence, privilege escalation, credential access, lateral movement, and impact. It does not cover the entire Enterprise ATT&CK matrix. Several tasks also depend on vulnerable software, disabled or reduced defenses, and pre-established access.

6. What the Evaluation Demonstrates

6.1 Experimental Environment

The authors built a Hyper-V testbed on a Windows 10 host. It included a Kali Linux attacker VM, Windows 10, Windows 11, Windows Server 2016, Ubuntu 12.04, and Ubuntu 22.04 virtual machines. The Windows Server VM served as a domain controller. The attacker VM included Metasploit. The researchers reset VMs at the start of each task and limited each task to 30 interactions. The victim network had no internet access; the host communicated with the LLM service [1].

This containment approach should be preserved and strengthened. The project should use disposable snapshots, outbound network controls, synthetic credentials, nonproduction data, and explicit target allowlists. Logs should capture every model call and executed action.

6.2 Reported Model Results

At temperature 0, Table III reports a 3/3 success rate for GPT-4 on 13 of 14 tasks. It lists Log4j Header Injection at 2/3. The surrounding text and conclusion state that GPT-4 achieved perfect success on every task. That internal discrepancy should prevent the project from repeating the paper's perfect-success claim without qualification [1].

For the successful GPT-4 trials at temperature 0, the paper reports average interaction counts ranging from 3 for file uploading to 17 for the ransomware task. Pass-the-hash averaged 14.3 interactions; malicious WinRAR averaged 13.6. Increased temperature reduced consistency on dump-the-hash, Log4j injection, and SSH public-key injection [1].

GPT-3.5 completed only file writing, privilege escalation, and MySQL scanning in at least one evaluated configuration. The Llama2-7B-chat and Llama2-70B-chat models completed none of the 14 tasks in the authors' setup [1]. The difference shows that a modular architecture does not compensate for weak tool knowledge, poor instruction following, or inaccurate state tracking.

6.3 Component Contribution

The ransomware ablation study provides direct evidence for the summarizer and experience manager. With both modules and an abstract objective, the system succeeded in 3/3 trials within 7 interactions. Without the experience manager, the abstract-objective variant failed all three trials. A detailed objective compensated for missing experience to some degree, reaching 3/3 with the summarizer alone at 15 interactions and 2/3 with neither module at 15 interactions [1].

The result supports procedural memory and context management. It does not establish that every retrieved action is correct, that the result generalizes to well-defended networks, or that the LLM can judge findings accurately.

6.4 Hallucination and Self-Correction

The paper documents hallucinated or invalid commands from every model class. GPT-4 sometimes proposed an incorrect Metasploit module but recovered after observing a failed command and searching the available modules. The Llama 2 models often generated nonexistent commands or ignored the required action format [1].

Self-correction is useful for navigation. It is insufficient for finding validation. A model can produce a plausible story after a failed command or misread a noisy result as success. The extension therefore separates exploration from adjudication.

7. Limits of the Current Evidence

The project should treat the paper as a strong feasibility study, not proof of production readiness.

- **Deliberately vulnerable targets.** The authors disabled or reduced protective controls, including antivirus in relevant scenarios. The results do not show performance against a hardened enterprise.
- **Small coverage.** Fourteen tasks cover only a small portion of MITRE ATT&CK.
- **Single testbed.** The benchmark uses one virtualized environment with specific operating systems, tool versions, and vulnerable services.
- **Model vintage.** The tested models and API configurations reflect 2023 and early 2024 capabilities. Current models require fresh evaluation.
- **Three trials per condition.** Three repetitions provide limited evidence about variance.
- **Researcher-authored objectives.** Detailed task descriptions can encode procedural knowledge that a fully autonomous system would otherwise need to discover.
- **LLM-based action selection.** The navigator relies on an LLM to choose among candidates; it does not independently prove that an action is valid or safe.
- **Summary loss.** The summarizer can omit details needed for later decisions.
- **No canonical code release identified.** The paper specifies implementation technologies but does not provide an official repository or software license.
- **Internal reporting inconsistency.** The text claims perfect GPT-4 success at temperature 0, while Table III lists one task at 2/3.

These limits define the opportunity. A course project can improve the validity, traceability, and reproducibility of the agent's output without attempting to solve the entire penetration-testing problem.

8. Proposed Reasoning-Layer Extension

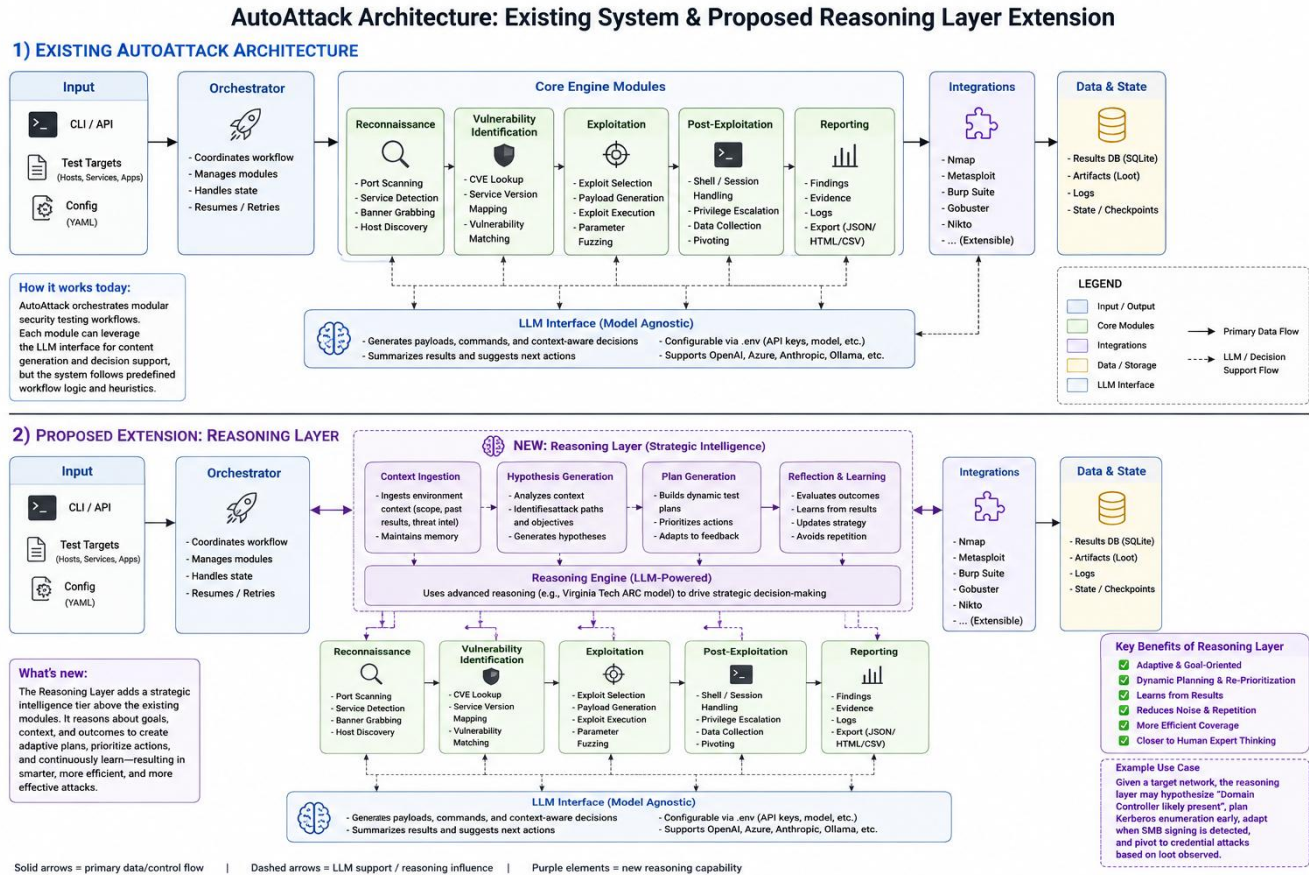


Figure 1: Conceptual project architecture for extending AUTOATTACKER with strategic reasoning. The figure is a target-state project illustration, not an as-built diagram of the 2024 research prototype.

The supplied architecture figure presents two views. The upper view describes a conventional modular security-testing workflow with reconnaissance, vulnerability identification, exploitation, post-exploitation, reporting, integrations, and state storage. The lower view adds context ingestion, hypothesis generation, plan generation, reflection, and learning. Purple elements represent the proposed reasoning capability.

The figure should be interpreted as a project vision. The cited AUTOATTACKER paper does not document all upper-view modules or listed integrations as implemented features. It documents a narrower LLM-controlled test loop centered on the summarizer, planner, navigator, experience manager, shell, Metasploit, and virtual test environment [1]. The extension can place that loop behind a broader orchestration and evidence interface.

8.1 Context Ingestion

Context ingestion creates the authoritative project state. It should combine the approved scope, target inventory, objective, available tools, prior observations, and safety constraints. Structured fields should include:

- authorized target identifiers and network ranges;
- allowed techniques, tools, and time windows;
- initial access assumptions and synthetic credentials;
- known services, operating systems, and defensive controls;

- task completion predicates and required proof;
- prohibited actions, rate limits, and stop conditions; and
- provenance for each fact, including whether it came from configuration, a tool observation, a human, or an LLM inference.

The context store should distinguish facts from hypotheses. A statement such as "port 445 responded at 10:03" is an observation. A statement such as "the host may be a domain controller" is a hypothesis until another source confirms it.

8.2 Hypothesis Generation

The reasoning engine proposes testable hypotheses from the current facts. Each hypothesis should include a rationale, required preconditions, proposed observation, expected evidence, validation rule, risk, and estimated cost. Examples include a suspected exposed service, a possible privilege boundary, or a candidate path between two lab systems.

The LLM may rank hypotheses, but it should not promote one to a finding. The orchestrator selects a bounded test only after checking scope, tool availability, and safety policy.

8.3 Plan Generation and Reprioritization

The plan generator converts hypotheses into a dependency graph rather than a single command sequence. Each step includes:

- action type and approved tool;
- target and parameter bindings;
- required state and expected observation;
- evidence collector and validator;
- timeout, retry policy, and abort condition;
- rollback or cleanup action; and
- next branches for success, failure, ambiguity, or unexpected output.

The planner should reprioritize when evidence changes. A failed precondition can invalidate a branch. A high-confidence observation can eliminate redundant tests. An ambiguous result can trigger a lower-risk confirmation step.

8.4 Reflection and Learning

Reflection evaluates the run after deterministic validation. It can identify repeated commands, unproductive hypotheses, incorrect environmental assumptions, and retrieved experiences that failed to transfer. Learning should update metadata about an experience rather than rewriting history. The system should retain both successes and failures, including the conditions under which each occurred.

The extension should never let the LLM mark its own plan as successful. It may explain the result and suggest a next action; the validator controls task status.

8.5 Reasoning Engine

The reasoning engine coordinates context, hypotheses, plans, and reflection. It can use an approved commercial model, a locally hosted model, or a model provided through Virginia Tech ARC. The interface should remain model-agnostic so the project can compare models without changing the execution and validation layers.

The engine should return structured objects validated against a schema. Free-form text can accompany the object for a human reviewer, but it should not directly drive tools.

8.6 Tool Integration Layer

The architecture figure lists Nmap, Metasploit, Burp Suite, Gobuster, and Nikto as possible integrations. These are proposed adapters, not documented AUTOATTACKER capabilities. A course implementation should begin with one or two tools that cover the selected benchmark tasks. Each adapter should expose:

- a typed action schema;
- target and parameter validation;
- an allowlist of permitted operations;
- standardized result parsing;
- raw output preservation;
- timeout and cancellation; and
- a validator that is independent of the LLM.

Tool adapters should not accept arbitrary model-generated shell strings by default. A typed interface makes unsafe or malformed operations easier to reject.

8.7 State, Evidence, and Reporting

The state layer should store the current plan, observations, actions, artifacts, and checkpoints. A separate evidence graph connects findings to proof. Each evidence node should record its source, collection time, target, tool version, raw output hash, and relationship to the task predicate.

Reporting should generate two views. The operator view explains the plan, current evidence, uncertainties, and next decision. The final report lists only verified findings, with reproduction conditions and remediation context. Failed hypotheses belong in the audit trail, not in the findings list.

9. Evidence-Grounded Finding Model

The central project requirement is that a finding must be real. The system needs an explicit evidence policy.

9.1 Evidence Classes

Class	Meaning	May appear as a finding?	Example treatment
Observation	Raw output or artifact collected from an approved tool	No	Preserve with timestamp, target, parameters, and hash.
Hypothesis	LLM or rule-based interpretation that predicts a condition	No	Schedule a bounded test or reject it.
Confirmed	Two compatible observations or one observation plus a deterministic predicate	Usually	Require the applicable finding rule.
Verified finding	Completion predicate satisfied, scope confirmed, and evidence bundle complete	Yes	Include in the report with severity and remediation.
Inconclusive	Evidence is missing, conflicting, stale, or nonrepeatable	No	Record uncertainty and the next safe check.
Invalid	Parser error, out-of-scope target, malformed action, or failed precondition	No	Stop or replan; do not count it as coverage.

9.2 Finding Contract

Every finding type should have a versioned contract defined before the test begins:

1. **Scope predicate.** The target is inside the approved range and the action is permitted.
2. **Preconditions.** The required service, session, privilege, or test fixture exists.
3. **Primary observation.** An approved tool produces the expected raw result.
4. **Confirmation rule.** A deterministic check or independent second observation confirms the condition.
5. **Success predicate.** The task objective is observable in the victim environment.
6. **Evidence bundle.** The system records tool version, sanitized command or action object, target, time, outputs, artifacts, and hashes.
7. **Cleanup check.** The system removes the test artifact or restores the VM snapshot.

A finding that lacks one of these elements remains inconclusive.

9.3 Independent Validation

The validator should not ask the same LLM that proposed an action whether the action succeeded. Suitable validators include:

- direct file, process, service, account, or privilege checks inside the range;
- a second tool that measures the same condition through a different mechanism;
- comparison with a seeded ground-truth manifest;
- target-side telemetry such as event logs or controlled sensors; and
- a repeat run from a clean snapshot when repeatability is required.

An LLM can summarize validator output after the status is set. It cannot override the validator.

9.4 Provenance and Replay

The evidence record should support replay. Each action receives a stable identifier. Observations reference the action that produced them. Findings reference the observations and validator decisions that support them. The record also preserves rejected actions, policy denials, retries, and model responses.

This structure allows a reviewer to answer four questions: what the agent believed, what it did, what the environment returned, and why the system called the result a finding.

10. Mapping the Extension to AUTOATTACKER

- **Task `t = (env, obj)`:** add scope policy, an explicit success predicate, an evidence contract, and a cleanup rule. The task becomes testable before the agent starts.
- **Summarizer:** preserve raw evidence and build structured state with provenance and uncertainty. The model cannot erase the only record through summarization.
- **Planner:** add a global plan graph, hypotheses, dependencies, priorities, and branch conditions. Local actions serve a visible strategy.
- **Command checker:** replace arbitrary command strings with typed tool calls and parameter validation. The gateway rejects malformed and unsafe actions.
- **Experience manager:** store preconditions, outcomes, confidence, failures, tool versions, and target context. Reuse becomes conditional and auditable.

- **Navigator:** add policy checks, expected-observation checks, and bounded execution. Action selection cannot bypass scope or safety constraints.
- **Victim environment:** add seeded ground truth, target-side telemetry, snapshots, and cleanup verification. Results can be measured against known state.
- **Interaction log:** build an evidence graph and replayable decision log. Findings remain connected to proof.

The project preserves AUTOATTACKER's useful tactical loop. It adds the controls required to treat the output as defensible security evidence.

11. Example Project Scenario

The supplied diagram gives a domain-controller scenario. The project can implement a safe version in an isolated Windows domain range:

- The operator provides a goal, approved targets, technique allowlist, and success predicate.
- Context ingestion records known hosts and current observations.
- The reasoning layer proposes several hypotheses about the path to the goal.
- The plan generator selects a low-risk information-gathering step and binds it to an approved adapter.
- The execution layer runs the step and records raw output.
- A validator compares the observation with target-side ground truth.
- The planner reprioritizes and may abandon a path if evidence contradicts an assumption.
- The system reports a finding only after the success predicate and confirmation rule pass.
- The range returns to a known snapshot.

The demonstration should visualize the plan and evidence state in real time. A reviewer should be able to select a finding and trace it backward to the validating observation, then to the action and plan decision that produced it.

12. Proposed Implementation Architecture

12.1 Services and Data Objects

The project can use a small set of services:

- **Orchestrator:** owns the run lifecycle, target scope, retries, timeouts, and checkpoints.
- **Reasoning service:** produces hypotheses, plans, and structured action proposals.
- **Policy engine:** enforces targets, techniques, tools, rates, and stop conditions.
- **Tool gateway:** validates typed actions and invokes approved adapters.
- **Observation normalizer:** parses results while preserving raw output.
- **Validator:** evaluates deterministic finding contracts and target-side ground truth.
- **Evidence store:** records immutable actions, observations, artifacts, and relationships.
- **Experience store:** retains conditional procedural experience for retrieval.
- **Reporter:** creates operator and final reports from verified records.

Core data objects should include Objective, Scope, Hypothesis, Plan, PlanStep, Action, Observation, ValidationDecision, Finding, Artifact, and Experience. Each object should carry a schema version and provenance fields.

12.2 Control Flow

The orchestrator starts with a validated objective and scope. The reasoning service proposes hypotheses and a plan. The policy engine checks each proposed action before the tool gateway executes it. The normalizer returns an observation. The validator evaluates the observation against the step's contract. The reasoning service receives the structured outcome, not an unbounded transcript. The process continues until the objective is verified, the plan is exhausted, the risk budget is reached, or a human stops the run.

12.3 Human Oversight

The project should provide three execution modes:

- **Observe:** the system plans and explains, but a human executes actions.
- **Approve:** the system waits for approval before selected action classes.
- **Autonomous range:** the system executes allowlisted actions only inside the disposable lab.

The autonomous mode should remain unavailable until the project passes scope, rollback, and evidence tests. High-impact actions require explicit approval even inside the range unless the course protocol approves them in advance.

13. Project Evaluation Plan

The evaluation should compare the baseline and extension under identical targets, starting states, task definitions, model settings, and action budgets.

13.1 Experimental Conditions

Condition	Description
Baseline	Reproduced AUTOATTACKER-style summarizer, planner, experience manager, navigator, and executor.
Reasoning only	Baseline plus hypotheses and an explicit revisable plan.
Evidence grounded	Reasoning condition plus deterministic validators, provenance, and finding contracts.
Ablation	Remove the global plan, validator, structured state, or experience metadata one at a time.

13.2 Research Questions

RQ1: Does the reasoning-layer extension increase verified task success? Compare the percentage of runs that satisfy the predeclared task predicate.

RQ2: Does independent validation reduce false findings? Seed the range with present and absent conditions. Compare reported findings with ground truth.

RQ3: Does explicit planning reduce wasted actions? Measure repeated commands, invalid actions, abandoned branches, and total interactions.

RQ4: Does provenance improve reproducibility? Give a second evaluator the evidence bundle and measure whether the evaluator can reproduce or reject the finding without the original chat transcript.

13.3 Metrics

Primary metrics should include:

- verified task success rate;
- finding precision, recall, and false-positive rate against seeded ground truth;
- percentage of findings with complete evidence bundles;
- number of tool actions and LLM interactions;
- repeated or policy-denied actions;
- time and model cost per verified objective;
- recovery rate after an injected tool error;
- run-to-run consistency from the same snapshot; and
- cleanup success after task completion or failure.

Secondary measures can include plan revisions, hypotheses eliminated by evidence, retrieval usefulness, human interventions, and report review time.

13.4 Task Selection

A feasible first experiment can use four tasks:

- one basic state-changing task with a direct target-side predicate;
- one multi-step Windows task that requires privilege or session tracking;
- one Linux service task that forces the agent to recover from a deliberately injected tool error; and
- one negative-control task where the suspected weakness is absent.

The negative control is essential. A system that reports only seeded-positive tasks cannot demonstrate that it avoids false findings.

13.5 Analysis

Use paired runs from identical VM snapshots. Fix model version, temperature, maximum interactions, and tool versions within an experimental block. Report every failed and timed-out run. Where sample size permits, compare paired binary outcomes with McNemar's test and compare action counts with a paired nonparametric test. Report effect sizes and confidence intervals. The course project can also provide descriptive results if the number of runs is too small for stable inference, but it should label them accordingly.

14. Implementation Plan

Phase 0: Baseline and Containment

- select and verify the codebase;
- reproduce one basic task in a disposable range;
- implement scope enforcement, snapshots, synthetic credentials, and immutable logging; and
- document the baseline architecture and known deviations from Xu et al.

Exit criterion: the team can repeat one task from a clean snapshot and obtain the same ground-truth result without relying on an LLM's self-assessment.

Phase 1: Evidence Model

- define the action, observation, validation, and finding schemas;
- build a target-side ground-truth manifest;
- implement validators for the selected tasks; and

- generate a finding report from verified records.

Exit criterion: the system distinguishes verified, inconclusive, and invalid outcomes on seeded positive and negative cases.

Phase 2: Strategic Reasoning

- add context ingestion and hypothesis objects;
- build a plan graph with dependencies and branch conditions;
- expose plan revisions and rejected hypotheses; and
- connect the local AUTOATTACKER planner to the selected plan step.

Exit criterion: a tool failure or contradicted assumption causes a recorded replan rather than blind repetition.

Phase 3: Conditional Experience and Tool Adapters

- enrich retrieved experiences with preconditions and outcome metadata;
- validate retrieved action parameters before execution;
- implement the minimum tool adapters needed for the experiment; and
- add cancellation, timeout, and cleanup hooks.

Exit criterion: retrieved experience improves at least one task without increasing false findings.

Phase 4: Controlled Evaluation

- execute paired baseline and extension runs;
- run ablations;
- calculate verified success, false findings, actions, time, and cost; and
- produce a reproducibility package containing configurations, schemas, sanitized logs, and range reset instructions.

Exit criterion: the final report supports each conclusion with recorded experimental evidence.

15. Safety, Ethics, and Governance

The project requires an explicit rules-of-engagement document. NIST SP 800-115 provides established guidance for planning and conducting authorized information security testing [4]. The NIST AI Risk Management Framework supplies a complementary structure for governing model risk, measurement, and oversight [5]. The following controls should be mandatory:

- written authorization that names owners, targets, techniques, dates, and stop conditions;
- a physically or logically isolated range with no route to production systems;
- default-deny egress, allowlisted model endpoints, and no production credentials;
- disposable VMs and automated snapshots;
- target and command validation outside the LLM;
- action-level audit logging with synchronized time;
- secrets redaction in model prompts and reports;
- human approval for destructive or high-impact actions;
- emergency stop and session revocation; and
- post-run cleanup and verification.

The range should contain synthetic data and synthetic identities. The project should not test third-party systems, public addresses, university production services, or employer networks without separate written authorization.

[TODO: attach the course-approved rules of engagement, target inventory, permitted ATT&CK techniques, and named approver before autonomous execution is enabled.]

16. Key Risks and Mitigations

Risk	Effect	Mitigation
Hallucinated command or module	Failed action or unsafe behavior	Typed adapters, command schemas, tool introspection, and policy checks.
Hallucinated finding	False report and loss of trust	Deterministic finding contracts and independent confirmation.
State-summary loss	Wrong target, shell, privilege, or path	Structured state plus immutable raw observations.
Experience mismatch	A prior action is reused in the wrong context	Store preconditions, versions, target type, and prior outcome; validate bindings.
Scope escape	Interaction with an unauthorized target	Network controls, target allowlist, gateway enforcement, and emergency stop.
Repetition loop	Excessive actions, cost, or detection	Action fingerprinting, retry budgets, novelty checks, and plan-level stop rules.
Destructive side effect	Loss of range state or unstable result	Snapshots, synthetic data, approval gates, and automatic cleanup.
Model drift	Results change across time	Pin model identifiers, archive prompts and schemas, and rerun benchmark controls.
Unsupported codebase	Project starts from an unrelated implementation	Confirm provenance, license, paper alignment, and baseline behavior before extension.

17. Recommended Minimum Viable Project

The minimum viable project should demonstrate a complete evidence path rather than a large attack catalog. A strong scope includes:

- one verified AUTOATTACKER-style baseline implementation;
- four laboratory tasks, including a negative control;
- a visible hypothesis and plan graph;
- two typed tool adapters;
- deterministic validators and target-side ground truth;
- an evidence graph and replayable report;
- paired baseline and extension runs; and
- a short ablation that removes validation or strategic planning.

This scope is large enough to test the central claim and small enough to complete in a semester. Additional ATT&CK techniques, web interfaces, model comparisons, and ARC-scale inference should follow only after the evidence path works end to end.

18. Conclusion

AUTOATTACKER demonstrates that a capable LLM can coordinate multi-step cyber-range actions when the system maintains context, retrieves prior experience, and executes one action at a time. Its 2024 evaluation also shows why an LLM should not serve as the final judge of security findings. Results depend heavily on the model, the environment is deliberately vulnerable, hallucinations occur, and the paper reports an internal discrepancy in one success table [1].

The proposed project adds strategic reasoning without trusting reasoning as proof. The LLM generates hypotheses and revises plans. Typed tool adapters constrain execution. Deterministic validators and target-side telemetry decide whether evidence supports a finding. A provenance graph connects every reported condition to the actions and observations that produced it.

That separation gives the project a clear contribution: an adaptive autonomous testing workflow that produces auditable findings instead of plausible guesses. The result can advance AUTOATTACKER from a feasibility prototype toward a defensible research platform for controlled, authorized security evaluation.

References

- [1] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li, "AutoAttacker: A Large Language Model Guided System to Implement Automatic Cyber-attacks," arXiv:2403.01038v1, March 2, 2024. <https://arxiv.org/html/2403.01038v1>.
- [2] MITRE, "MITRE ATT&CK: Enterprise Matrix." <https://attack.mitre.org/matrices/enterprise/>.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [4] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, *Technical Guide to Information Security Testing and Assessment*, NIST Special Publication 800-115, September 2008. <https://csrc.nist.gov/pubs/sp/800/115/final>.
- [5] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, January 2023. <https://doi.org/10.6028/NIST.AI.100-1>.
- [6] binarybrain-009, "AUTOATTACKER IMPLEMENTATION," public GitHub repository, accessed August 23, 2026. This repository is a third-party implementation and is not identified in Xu et al. as the authors' official release. <https://github.com/binarybrain-009/AUTOATTACKER>.